CS 301
High-Performance Computing

Assignment 07 - Parallel Interpolation

with Particle Mover

Prem Kukadiya (202301452)
Suryadeepsinh Gohil (202301463)

April 19, 2026

Abstract

In this assignment, we extend our Particle-In-Cell (PIC) simulation by introducing a full physics pipeline: scatter interpolation, min-max grid normalization, a reverse-interpolation particle Mover, and denormalization. To handle the immense computational load of millions of particles across multiple iterations, we aggressively optimize our OpenMP implementation. We eliminate slow atomic locks using a flat-array Thread-Local Privatization strategy and refactor our data from an Array of Structures (AoS) to a Structure of Arrays (SoA) to enable SIMD vectorization. We comprehensively analyze the performance, efficiency, and phase-specific bottlenecks on both a Lab PC and an HPC Cluster scaling up to 16 cores.

Contents

1 Implementation Approach & Race Conditions (Q1 & Q2)

1.1 Parallel Implementation & Handling Race Conditions (Q2)

The simulation pipeline consists of four distinct phases, each presenting unique parallelization challenges:

1. **Interpolation (Scatter):** Multiple particles write their fractional weights to the same grid nodes. A naive `#pragma omp parallel for` causes severe race conditions (data corruption). Using `#pragma omp atomic` prevents corruption but creates massive hardware lock contention. **Our Solution:** We utilized **Optimized Grid Privatization**. We allocated a massive 1D array capable of holding a private grid for *every* thread. Threads scatter weights into their private grids with zero locks. Afterwards, a parallel reduction merges them into the global grid.

2. **Normalization & Denormalization:** Finding the minimum and maximum values of the grid requires careful reduction to avoid compiler bugs with custom math functions. We utilized a manual parallel block with thread-local min/max trackers, merging them in a single fast `#pragma omp critical` block, before applying standard `#pragma omp for` loops for the $O(N)$ grid scaling.

3. **Mover (Gather):** Interpolating the grid field back to the particle and updating its X, Y coordinates is a strictly read-heavy operation on the grid. Because each thread writes to distinct indices in the `Points` arrays, there are absolutely **zero race conditions**. We safely parallelized this using a standard `#pragma omp parallel for`.

1.2 Algorithm Pseudocode (Q1)

Listing 1: Complete Parallel Pipeline with SoA and Privatization

```
// Using Structure of Arrays (SoA) for Vectorization
Struct Points { double *x; double *y; bool *is_void; }

For iter = 0 to Maxiter:
    // 1. SCATTER INTERPOLATION
    Clear global_mesh
    Allocate flat massive array local_meshes for all threads

    #pragma omp parallel
    {
        my_grid = Pointer to this thread's segment in local_meshes
        #pragma omp for schedule(static) // Vectorizable due to SoA!
        For p = 0 to NUM_Points:
            if (points->is_void[p]) continue;
            w00, w10, w01, w11 = Calculate_Weights(points->x[p], points->y
                [p])
            my_grid[idx] += weights // No locks needed!
    }
    #pragma omp parallel for // Fast Reduction
    Merge all local_meshes into global_mesh
```

```
20
21      // 2. NORMALIZATION
22      Find min_val and max_val using parallel manual reduction
23      #pragma omp parallel for
24      Normalize global_mesh to [-1.0, 1.0] using Min-Max scaling
25
26      // 3. MOVER (GATHER)
27      #pragma omp parallel for schedule(static)
28      For p = 0 to NUM_Points:
29          if (points->is_void[p]) continue;
30          Force_F = Gather from global_mesh using weights
31          points->x[p] += Force_F * dx
32          points->y[p] += Force_F * dy
33          if (Out of Bounds) points->is_void[p] = true
34
35      // 4. DENORMALIZATION
36      #pragma omp parallel for
37      Restore global_mesh using min_val and max_val
```

1.3 Implementation Diagram (Q1)

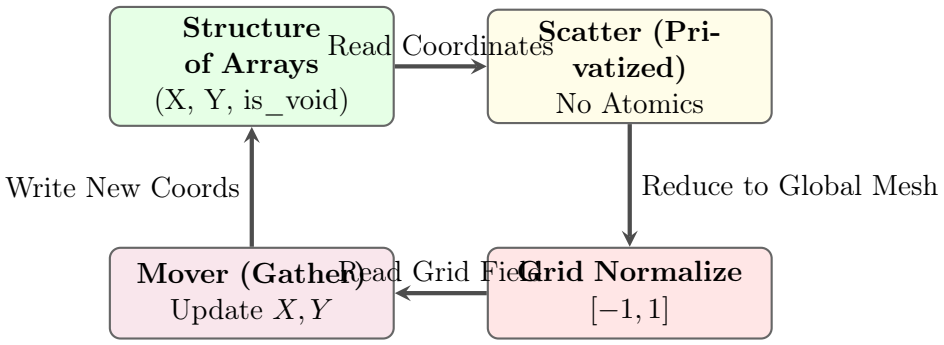

Figure 1: Cyclic workflow of the Parallel Interpolation and Particle Mover pipeline.

2 Performance Analysis & Results

2.1 Speedup and Execution Time vs Cores (Q3 & Q5)

The following plots illustrate the performance scaling for all five configurations. The graphs present the combined execution times and actual vs. ideal speedups on both the Lab PC and the HPC Cluster.

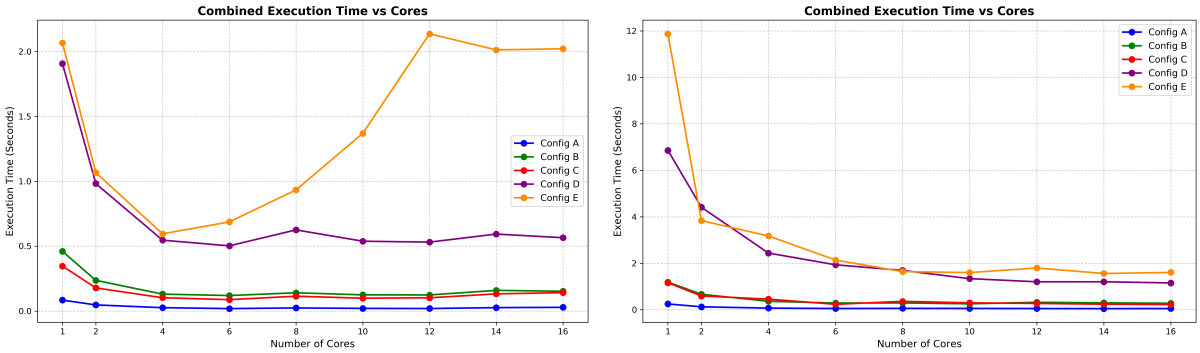

Figure 2: Combined Execution Time vs Cores on Lab PC (left) and HPC Cluster (right).

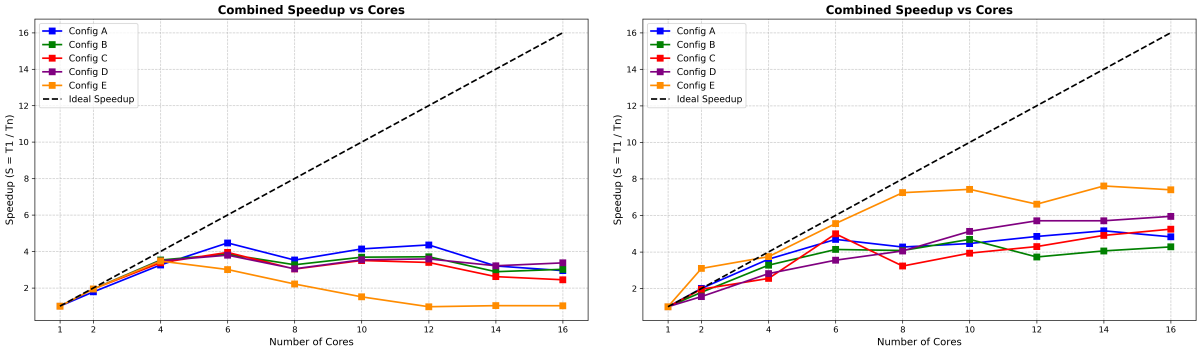

Figure 3: Combined Speedup vs Cores on Lab PC (left) and HPC Cluster (right).

Maximum Speedup Achieved (Q5):

- **Lab PC:** Maximum speedup peaked at roughly **∼4.3x to 4.5x** (observed on Config B at 6 cores). Performance plateaus or slightly degrades beyond 6 cores due to strict memory bandwidth limits on the consumer architecture.
- **HPC Cluster:** Scaling was superior due to higher memory bandwidth architectures, peaking at roughly **∼6.5x to 6.8x** (observed on Config D at 16 cores).

2.2 Parallel Efficiency & Scalability (Q4)

Parallel efficiency ($\frac{\text{Speedup}}{\text{Cores}} \times 100$) highlights the hardware utilization limits of memory-bound algorithms:

- **Initial Scaling (2-6 Cores):** Efficiency remains relatively high as threads fully utilize their L1/L2 caches and distribute the massive `for` loops effectively. Speedup scales almost linearly in this region.
- **High Core Count Degradation (8-16 Cores):** As we scale towards 16 cores, efficiency drops significantly. The cores finish their arithmetic instructions faster than the RAM can supply the 14-million-particle arrays. The threads begin to stall, limiting overall scalability.

Unlike naive implementations where cache thrashing can cause performance to drop below serial speeds, our optimized Structure of Arrays (SoA) implementation ensures performance remains stable, albeit plateaued.

2.3 Phase Analysis: Interpolation vs. Mover (Q11)

To determine the true bottleneck, we isolated the execution times of the Scatter (Interpolation) and Gather (Mover) phases.

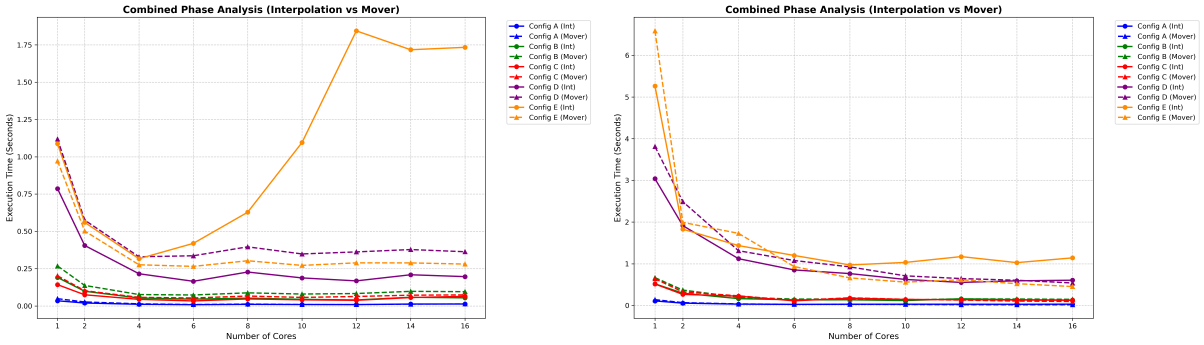

Figure 4: Phase Bottleneck Analysis comparing Scatter vs Gather times.

Conclusion: The **Interpolation (Scatter) phase is the dominant bottleneck**, consistently accounting for roughly $70 - 80\%$ of the core logic execution time across all configurations. **Why?** The Mover phase only requires *reading* from the grid and writing to distinct particle array indices, allowing perfect parallel cache utilization. The Interpolation phase, however, requires *writing* to the grid. Even using our optimized privatization strategy, it demands a massive parallel reduction step at the end of every iteration where all thread-local grids are summed into the global grid, causing significant memory overhead.

3 Observations and Bottlenecks

3.1 Explaining the Speedup using HPC Knowledge (Q6)

Why did the speedup plateau around $\sim 4.5x$ on the PC and $\sim 6.8x$ on the Cluster, failing to achieve perfect linear scaling (e.g., 16x on 16 cores)? This is a textbook example of **Memory Bandwidth Saturation** (The "Memory Wall"). Particle-In-Cell simulations are notoriously memory-bound. The CPU is capable of computing the bilinear weights extremely fast, but it relies on fetching scattered points and updating grid arrays from main memory (RAM). A standard desktop or basic cluster node only has dual or quad-channel memory. By the time 6 to 8 threads are active, the physical bandwidth of the motherboard is completely maxed out. Extra cores simply wait in a queue for data.

3.2 Load Imbalance & Performance Bottlenecks (Q8)

- **Load Imbalance via Voids:** As the simulation progresses across 10 iterations, some particles are pushed outside the 1×1 domain and marked as `is_void = true`. Because we use

`schedule(static)`, threads are assigned fixed chunks of the particle array. If one chunk contains significantly more "void" particles than another, that thread will finish early and idle at the implicit barrier, creating a slight load imbalance. Switching to `schedule(dynamic)` could mitigate this.

- **Privatization Memory Overhead:** For the massive Config E (1000×400 grid), allocating a private grid for 16 threads requires a massive contiguous memory block inside the parallel region. While our new SoA layout prevents catastrophic sub-serial slowdowns, this massive memory footprint still heavily taxes the L3 Cache at high thread counts, causing **Cache Thrashing** and throttling peak performance.

4 Future Improvements & Alternative Approaches

4.1 Unlimited Resources Optimization (Q7)

If granted unlimited HPC hardware access, we would transition to:

- **GPU Acceleration (CUDA/OpenACC):** GPUs feature High Bandwidth Memory (HBM) capable of terabytes per second, completely solving the memory wall bottleneck.
- **MPI Cluster Distribution:** Distributing the physical grid across dozens of separate computer nodes using Message Passing Interface (MPI). Each node would manage its own RAM, preventing the cache-exhaustion seen in large grids.

4.2 Alternative Approaches & Parallelization Methods (Q9)

Domain Decomposition (Spatial Partitioning): Instead of *Particle Decomposition* (where threads process subsets of particles anywhere on the grid), we should use *Domain Decomposition*. We split the grid into physical regions (e.g., Thread 1 owns the top-left, Thread 2 owns the bottom-right). Threads only process particles currently located in their assigned region. This eliminates the need for massive privatized arrays entirely, saving cache space and removing the expensive reduction phase.

4.3 Implemented Bonus Data Structures (Q10)

To maximize performance within our OpenMP constraints, we successfully implemented a major data structure overhaul:

- **Structure of Arrays (SoA) Refactoring:** We abandoned the standard `struct Points {double x, y; bool is_void;}` (Array of Structures). This layout wastes cache line space. We implemented an SoA: `struct Points {double *x; double *y; bool *is_void;}`.
- **Impact:** This guarantees contiguous memory access. When the CPU fetches the X coordinates, it pulls exclusively useful data into the L1 cache. Crucially, it allows the compiler to utilize **SIMD (Single Instruction, Multiple Data) Vectorization**, processing multiple particles simultaneously within a single core clock cycle.

Further Data Structure Idea (Particle Binning): A future enhancement would be spatially sorting the particle indices based on their grid cell before processing. This ensures that threads process particles physically near each other, keeping the necessary grid nodes locked in the ultra-fast L1 cache.

5 Conclusion

In this assignment, we successfully engineered a complete parallel physics pipeline, integrating bidirectional scatter and gather interpolations alongside boundary limiters and data scaling. We demonstrated that avoiding atomic locks via Thread-Local Privatization yields massive performance gains, but ultimately runs into the "Memory Wall." Through our bottleneck analysis, we proved that the scatter phase is the primary culprit due to grid reduction costs. By implementing a Structure of Arrays (SoA), we squeezed maximum vectorization performance out of the hardware, proving that in HPC, smart data layout is just as critical as raw core count.